Digital System Design Assignment

A.H.T.M.Weerakoon
230689A

July 19, 2026

Contents

1 Exercise 1 : Sequence Detector (0110)

1.1 Objective

Design a sequence detector that detects the binary sequence **0110** for

- Overlapping detection
- Non-overlapping detection

1.2 Verilog Code

```
1
2  module seq_detector_overlap(
3      input logic clk,
4      input logic rst,
5      input logic x,
6      output logic y
7  );
8
9  typedef enum logic [2:0] {
10     S0, S1, S2, S3, S4
11 } state_t;
12
13 state_t state, next_state;
14
15 // State register
16 always_ff @(posedge clk or posedge rst) begin
17     if (rst)
18         state <= S0;
19     else
20         state <= next_state;
21 end
22
23 // Next-state logic
24 always_comb begin
25     case (state)
26
27         S0: begin
28             if (x)
29                 next_state = S0;
30             else
31                 next_state = S1;
32         end
33
34         S1: begin
35             if (x)
36                 next_state = S2;
37             else
38                 next_state = S1;
39         end
```

```
40
41          S2: begin
42              if (x)
43                  next_state = S3;
44              else
45                  next_state = S1;
46          end
47
48          S3: begin
49              if (x)
50                  next_state = S0;
51              else
52                  next_state = S4;
53          end
54
55          S4: begin
56              if (x)
57                  next_state = S2;
58              else
59                  next_state = S1;
60          end
61
62          default: next_state = S0;
63
64      endcase
65  end
66
67  // Output logic
68  assign y = (state == S4);
69
70  endmodule
```

Listing 1: Sequence Detector With Overlap

```
1
2  `timescale 1ns/1ps
3
4  module testbench;
5
6  logic clk;
7  logic rst;
8  logic x;
9  logic y;
10
11 seq_detector_overlap uut (
12     .clk(clk),
13     .rst(rst),
14     .x(x),
15     .y(y)
16 );
17
```

```
// Clock generation
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    x = 0;

    #10 rst = 0;

    // Input sequence: 0 1 1 0 1 1 0
    x = 0; #10;
    x = 1; #10;
    x = 1; #10;
    x = 0; #10;
    x = 1; #10;
    x = 1; #10;
    x = 0; #10;

    #20;
    $finish;
end

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, testbench);
end

endmodule
```

Listing 2: Testbench

```

module seq_detector_non_overlap(
    input logic clk,
    input logic rst,
    input logic x,
    output logic y
);

typedef enum logic [2:0] {
    S0, S1, S2, S3, S4
} state_t;

state_t state, next_state;

// State register
always_ff @(posedge clk or posedge rst) begin
    if (rst)
        state <= S0;
    else
        state <= next_state;
```

```
end

// Next-state logic
always_comb begin
    case (state)

        S0: begin
            if (x)
                next_state = S0;
            else
                next_state = S1;
        end

        S1: begin
            if (x)
                next_state = S2;
            else
                next_state = S1;
        end

        S2: begin
            if (x)
                next_state = S3;
            else
                next_state = S1;
        end

        S3: begin
            if (x)
                next_state = S0;
            else
                next_state = S4;
        end

        S4: begin
            if (x)
                next_state = S0;
            else
                next_state = S1;
        end

        default: next_state = S0;

    endcase
end

// Output logic
assign y = (state == S4);

endmodule
```

Listing 3: Sequence Detector Without Overlap

```
1
2  `timescale 1ns/1ps
3
4  module testbench;
5
6  logic clk;
7  logic rst;
8  logic x;
9  logic y;
10
11 seq_detector_non_overlap uut (
12     .clk(clk),
13     .rst(rst),
14     .x(x),
15     .y(y)
16 );
17
18 // Clock generation
19 always #5 clk = ~clk;
20
21 initial begin
22     clk = 0;
23     rst = 1;
24     x = 0;
25
26     #10 rst = 0;
27
28     // Input sequence: 0 1 1 0 1 0 1 1 0
29     x = 0; #10;
30     x = 1; #10;
31     x = 1; #10;
32     x = 0; #10;
33     x = 1; #10;
34   x = 0; #10;
35     x = 1; #10;
36     x = 1; #10;
37     x = 0; #10;
38
39     #20;
40     $finish;
41 end
42
43 initial begin
44     $dumpfile("dump.vcd");
45     $dumpvars(0, testbench);
46 end
47
48 endmodule
```

Listing 4: Testbench

1.3 Simulation Result

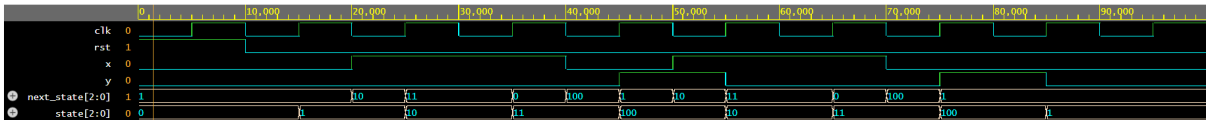

Figure 1: Simulation waveform for the overlapping sequence detector.

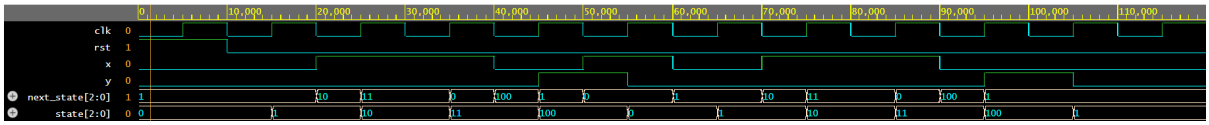

Figure 2: Simulation waveform for the non-overlapping sequence detector.

1.4 Maximum Supported Bit Rate

The maximum bit rate depends on the maximum operating clock frequency of the implemented design.

If

$$f_{max}$$

is the maximum clock frequency, then

$$\boxed{\text{Maximum Bit Rate} = f_{max} \text{ bits/s}}$$

because one input bit is processed during every clock cycle.
For example,

$$f_{max} = 100\,MHz$$

gives

$$\boxed{100\ Mbit/s}$$

The actual value depends on synthesis and FPGA timing analysis.

2 Exercise 2 : 32-bit Ripple Adder Using Four 8-bit Adders

2.1 Objective

Construct a 32-bit adder using four cascaded 8-bit ripple carry adders. Estimate the time required to add four pairs of 32-bit numbers. Then redesign the system using pipelining.

2.2 Verilog Code

```
1
2  `timescale 1ns/1ps
3
4  // 8-bit adder block
5  module adder8(
6      input  logic [7:0] A,
7      input  logic [7:0] B,
8      input  logic Cin,
9      output logic [7:0] Sum,
10     output logic Cout
11 );
12
13     assign {Cout, Sum} = A + B + Cin;
14
15 endmodule
16
17
18 // 32-bit adder using four cascaded 8-bit adders
19 module adder32_cascade(
20     input  logic [31:0] A,
21     input  logic [31:0] B,
22     output logic [31:0] Sum,
23     output logic Cout
24 );
25
26     logic c1, c2, c3;
27
28
29     adder8 add0(
30         .A(A[7:0]),
31         .B(B[7:0]),
32         .Cin(1'b0),
33         .Sum(Sum[7:0]),
34         .Cout(c1)
35     );
36
37
38     adder8 add1(
39         .A(A[15:8]),
40         .B(B[15:8]),
```

```
        .Cin(c1),
        .Sum(Sum[15:8]),
        .Cout(c2)
    );


    adder8 add2(
        .A(A[23:16]),
        .B(B[23:16]),
        .Cin(c2),
        .Sum(Sum[23:16]),
        .Cout(c3)
    );


    adder8 add3(
        .A(A[31:24]),
        .B(B[31:24]),
        .Cin(c3),
        .Sum(Sum[31:24]),
        .Cout(Cout)
    );

endmodule
```

Listing 5: 32-bit adder using four cascaded 8-bit adders

```

`timescale 1ns/1ps

module testbench;

    logic [31:0] A;
    logic [31:0] B;

    logic [31:0] Sum;
    logic Cout;


    // Instantiate design
    adder32_cascade DUT(
        .A(A),
        .B(B),
        .Sum(Sum),
        .Cout(Cout)
    );


    initial begin

        $dumpfile("dump.vcd");
```

```
        $dumpvars(0,testbench);


        A = 32'd100;
        B = 32'd50;

        #10;

        $display("A = %b",A);
        $display("B = %b",B);
        $display("SUM = %b",Sum);
        $display("CARRY = %b",Cout);


        A = 32'b11111111111111111111111111111110;
        B = 32'b10000000000000000000000000000001;

        #10;

        $display("SUM = %b",Sum);
        $display("CARRY = %b",Cout);


        #10;
        $finish;

    end

endmodule
```

Listing 6: Testbench

```

`timescale 1ns/1ps


module adder8(
    input  logic [7:0] A,
    input  logic [7:0] B,
    input  logic Cin,

    output logic [7:0] Sum,
    output logic Cout
);

    assign {Cout,Sum} = A + B + Cin;

endmodule



module adder32_pipeline(
```

```
21
22      input logic clk,
23      input logic rst,
24
25      input logic [31:0] A,
26      input logic [31:0] B,
27
28      output logic [31:0] Sum,
29      output logic Cout
30  );
31
32
33  ////////////////////////////////////////////////
34  // PIPELINE STAGE 1
35  // Add bits 7:0
36  ////////////////////////////////////////////////
37
38  logic [7:0] s0;
39  logic c0;
40
41
42  adder8 add0(
43      .A(A[7:0]),
44      .B(B[7:0]),
45      .Cin(1'b0),
46      .Sum(s0),
47      .Cout(c0)
48  );
49
50
51  logic [7:0] sum0_reg;
52  logic carry0_reg;
53
54  logic [23:0] A1_reg;
55  logic [23:0] B1_reg;
56
57
58  always_ff @(posedge clk)
59  begin
60
61      sum0_reg   <= s0;
62      carry0_reg <= c0;
63
64      A1_reg <= A[31:8];
65      B1_reg <= B[31:8];
66
67  end
68
69
70
71  ////////////////////////////////////////////////
```

```
72 // PIPELINE STAGE 2
73 // Add bits 15:8
74 ////////////////////////////////////////////////
75
76 logic [7:0] s1;
77 logic c1;
78
79
80 adder8 add1(
81     .A(A1_reg[7:0]),
82     .B(B1_reg[7:0]),
83     .Cin(carry0_reg),
84     .Sum(s1),
85     .Cout(c1)
86 );
87
88
89 logic [15:0] sum1_reg;
90
91 logic carry1_reg;
92
93 logic [15:0] A2_reg;
94 logic [15:0] B2_reg;
95
96
97 always_ff @(posedge clk)
98 begin
99
100     sum1_reg <= {s1,sum0_reg};
101
102     carry1_reg <= c1;
103
104     A2_reg <= A1_reg[23:8];
105     B2_reg <= B1_reg[23:8];
106
107 end
108
109
110
111 ////////////////////////////////////////////////
112 // PIPELINE STAGE 3
113 // Add bits 23:16
114 ////////////////////////////////////////////////
115
116 logic [7:0] s2;
117 logic c2;
118
119
120 adder8 add2(
121     .A(A2_reg[7:0]),
122     .B(B2_reg[7:0]),
```

```
    .Cin(carry1_reg),
    .Sum(s2),
    .Cout(c2)
);


logic [23:0] sum2_reg;

logic carry2_reg;

logic [7:0] A3_reg;
logic [7:0] B3_reg;


always_ff @(posedge clk)
begin

    sum2_reg <= {s2,sum1_reg};

    carry2_reg <= c2;

  A3_reg <= A2_reg[31:23];
  B3_reg <= B2_reg[31:23];

end



////////////////////////////////////////////////
// PIPELINE STAGE 4
// Add bits 31:24
////////////////////////////////////////////////

logic [7:0] s3;
logic c3;


adder8 add3(
    .A(A3_reg),
    .B(B3_reg),
    .Cin(carry2_reg),
    .Sum(s3),
    .Cout(c3)
);



////////////////////////////////////////////////
// OUTPUT REGISTER
////////////////////////////////////////////////

```

```
always_ff @(posedge clk)
begin

    if(rst)
    begin
        Sum  <= 0;
        Cout <= 0;
    end

    else
    begin
        Sum  <= {s3,sum2_reg};
        Cout <= c3;
    end

end


endmodule
```

Listing 7: 32-bit adder using pipeline

```

`timescale 1ns/1ps

module testbench;

logic clk;
logic rst;

logic [31:0] A;
logic [31:0] B;

logic [31:0] Sum;
logic Cout;


adder32_pipeline DUT(
    .clk(clk),
    .rst(rst),
    .A(A),
    .B(B),
    .Sum(Sum),
    .Cout(Cout)
);


always #5 clk = ~clk;


initial begin
```

```
30
31      $dumpfile("dump.vcd");
32      $dumpvars(0,testbench);
33
34
35      clk = 0;
36      rst = 1;
37
38      A = 0;
39      B = 0;
40
41
42      #12;
43      rst = 0;
44
45
46      // Apply input before clock edge
47
48      #3;
49      A = 100;
50      B = 50;
51
52
53      #10;
54      A = 200;
55      B = 300;
56
57
58      #10;
59      A = 1000;
60      B = 2000;
61
62
63      #10;
64      A = 4000;
65      B = 5000;
66
67
68      #10;
69      A = 0;
70      B = 0;
71
72
73      // Wait for pipeline output
74
75      repeat(6)
76      begin
77          @(posedge clk);
78
79          #1;
80
```

```
81          $display(
82          "Time=%0t  A=%d B=%d Sum=%d Carry=%b",
83          $time,A,B,Sum,Cout
84          );
85
86      end
87
88
89      $finish;
90
91  end
92
93
94  endmodule
```

Listing 8: Testbench

2.3 Simulation Result

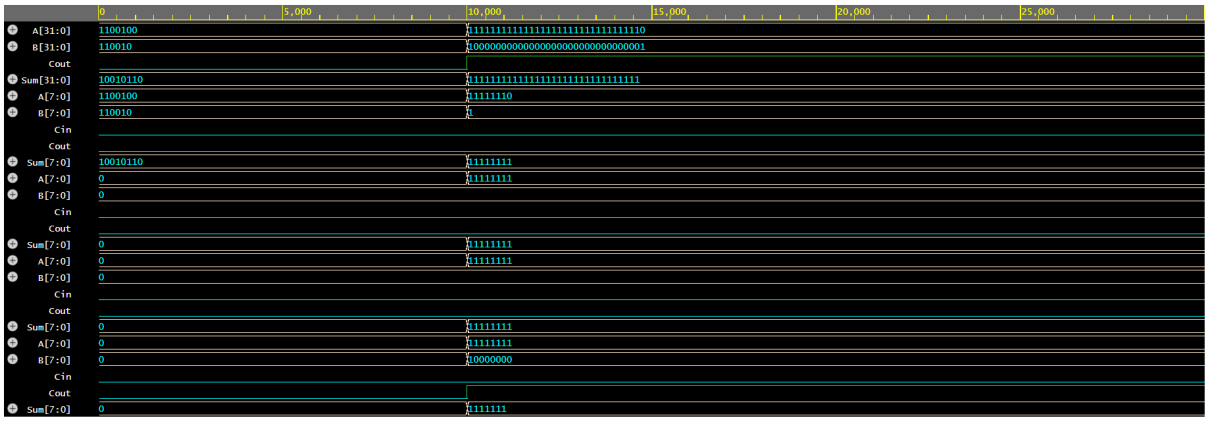

Figure 3: Simulation waveform for the 32-bit adder using four cascaded 8-bit adders

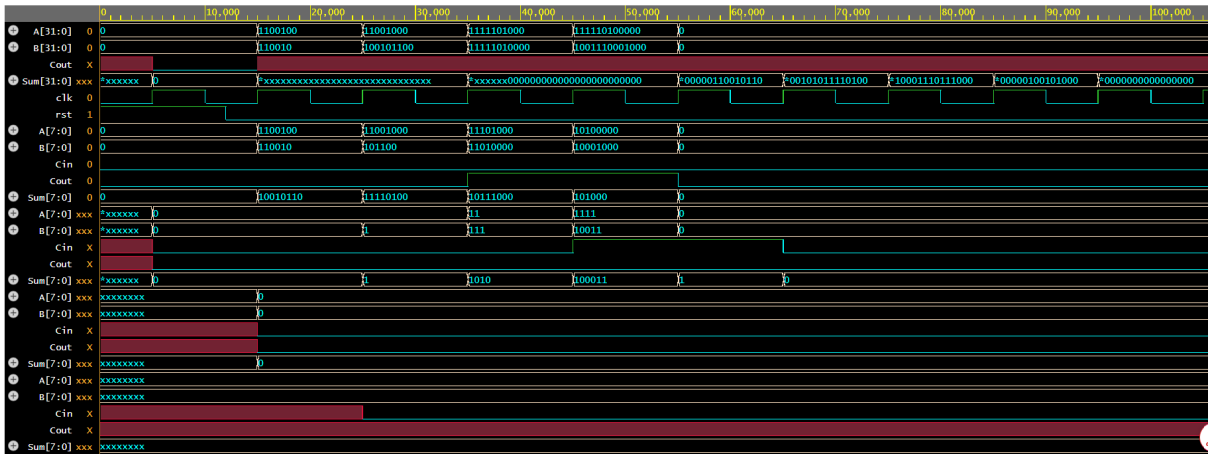

Figure 4: Simulation waveform for the 32-bit adder using pipeline

2.4 Time Required for Four Additions

Assume

$$T_8$$

is the propagation delay of one 8-bit adder.
Since four 8-bit adders are cascaded,

$$T_{32} = 4T_8$$

For four independent additions,

$$T_{Total} = 4T_{32}$$

Therefore,

$$\boxed{T_{Total} = 16T_8}$$

2.5 Pipelined 32-bit Adder

The pipeline divides the 32-bit addition into four stages.Each stage performs one 8-bit addition.
After the pipeline is filled,

- One completed 32-bit addition is produced every clock cycle.
- Multiple input pairs are processed simultaneously.
- Throughput increases significantly.

For four independent additions,

$$T_{Total} = 4T_8 + 3T_8$$

Therefore,

$$\boxed{T_{Total} = 7T_8}$$

2.6 Advantages of Pipelining

- Higher throughput
- Better hardware utilization
- Reduced critical path
- Higher maximum clock frequency

3 Exercise 3 : 32-bit Carry Lookahead Adder

3.1 Objective

Build a 32-bit Carry Lookahead Adder using four 8-bit CLA blocks.Compare its performance with the cascaded ripple carry adder.

3.2 Verilog Code

```
1
2  //============================================================
3  // 8-bit Carry Lookahead Adder
4  //============================================================
5  module cla8 (
6      input  [7:0] A,
7      input  [7:0] B,
8      input        Cin,
9      output [7:0] Sum,
10     output       Cout,
11     output       P_block,
12     output       G_block
13 );
14
15 wire [7:0] P, G;
16 wire [8:0] C;
17
18 assign C[0] = Cin;
19
20 // Propagate and Generate
21 assign P = A ^ B;
22 assign G = A & B;
23
24 // Carry Lookahead Equations
25 assign C[1] = G[0] | (P[0] & C[0]);
26
27 assign C[2] = G[1]
28             | (P[1] & G[0])
29             | (P[1] & P[0] & C[0]);
30
31 assign C[3] = G[2]
32             | (P[2] & G[1])
33             | (P[2] & P[1] & G[0])
34             | (P[2] & P[1] & P[0] & C[0]);
35
36 assign C[4] = G[3]
37             | (P[3] & G[2])
38             | (P[3] & P[2] & G[1])
39             | (P[3] & P[2] & P[1] & G[0])
40             | (P[3] & P[2] & P[1] & P[0] & C[0]);
41
42 assign C[5] = G[4]
```

```
43              | (P[4] & G[3])
44              | (P[4] & P[3] & G[2])
45              | (P[4] & P[3] & P[2] & G[1])
46              | (P[4] & P[3] & P[2] & P[1] & G[0])
47              | (P[4] & P[3] & P[2] & P[1] & P[0] & C[0]);
48
49  assign C[6] = G[5]
50              | (P[5] & G[4])
51              | (P[5] & P[4] & G[3])
52              | (P[5] & P[4] & P[3] & G[2])
53              | (P[5] & P[4] & P[3] & P[2] & G[1])
54              | (P[5] & P[4] & P[3] & P[2] & P[1] & G[0])
55              | (P[5] & P[4] & P[3] & P[2] & P[1] & P[0] & C[0]);
56
57  assign C[7] = G[6]
58              | (P[6] & G[5])
59              | (P[6] & P[5] & G[4])
60              | (P[6] & P[5] & P[4] & G[3])
61              | (P[6] & P[5] & P[4] & P[3] & G[2])
62              | (P[6] & P[5] & P[4] & P[3] & P[2] & G[1])
63              | (P[6] & P[5] & P[4] & P[3] & P[2] & P[1] & G[0])
64              | (P[6] & P[5] & P[4] & P[3] & P[2] & P[1] & P[0] & C
    [0]);
65
66  assign C[8] = G[7]
67              | (P[7] & G[6])
68              | (P[7] & P[6] & G[5])
69              | (P[7] & P[6] & P[5] & G[4])
70              | (P[7] & P[6] & P[5] & P[4] & G[3])
71              | (P[7] & P[6] & P[5] & P[4] & P[3] & G[2])
72              | (P[7] & P[6] & P[5] & P[4] & P[3] & P[2] & G[1])
73              | (P[7] & P[6] & P[5] & P[4] & P[3] & P[2] & P[1] & G
    [0])
74              | (P[7] & P[6] & P[5] & P[4] & P[3] & P[2] & P[1] & P
    [0] & C[0]);
75
76  assign Sum = P ^ C[7:0];
77  assign Cout = C[8];
78
79  // Block Propagate
80  assign P_block = &P;
81
82  // Block Generate
83  assign G_block =
84        G[7]
85      | (P[7]&G[6])
86      | (P[7]&P[6]&G[5])
87      | (P[7]&P[6]&P[5]&G[4])
88      | (P[7]&P[6]&P[5]&P[4]&G[3])
89      | (P[7]&P[6]&P[5]&P[4]&P[3]&G[2])
90      | (P[7]&P[6]&P[5]&P[4]&P[3]&P[2]&G[1])
```

```
    | (P[7]&P[6]&P[5]&P[4]&P[3]&P[2]&P[1]&G[0]);

endmodule

//============================================================
// 32-bit Carry Lookahead Adder (4    8-bit CLA)
//============================================================
module cla32 (
    input  [31:0] A,
    input  [31:0] B,
    input         Cin,
    output [31:0] Sum,
    output        Cout
);

wire [3:0] P, G;
wire [4:0] C;

assign C[0] = Cin;

// Lookahead carries between 8-bit blocks
assign C[1] = G[0] | (P[0] & C[0]);

assign C[2] = G[1]
            | (P[1] & G[0])
            | (P[1] & P[0] & C[0]);

assign C[3] = G[2]
            | (P[2] & G[1])
            | (P[2] & P[1] & G[0])
            | (P[2] & P[1] & P[0] & C[0]);

assign C[4] = G[3]
            | (P[3] & G[2])
            | (P[3] & P[2] & G[1])
            | (P[3] & P[2] & P[1] & G[0])
            | (P[3] & P[2] & P[1] & P[0] & C[0]);

cla8 CLA0(
    .A(A[7:0]),
    .B(B[7:0]),
    .Cin(C[0]),
    .Sum(Sum[7:0]),
    .Cout(),
    .P_block(P[0]),
    .G_block(G[0])
);

cla8 CLA1(
    .A(A[15:8]),
    .B(B[15:8]),
```

```
    .Cin(C[1]),
    .Sum(Sum[15:8]),
    .Cout(),
    .P_block(P[1]),
    .G_block(G[1])
);

cla8 CLA2(
    .A(A[23:16]),
    .B(B[23:16]),
    .Cin(C[2]),
    .Sum(Sum[23:16]),
    .Cout(),
    .P_block(P[2]),
    .G_block(G[2])
);

cla8 CLA3(
    .A(A[31:24]),
    .B(B[31:24]),
    .Cin(C[3]),
    .Sum(Sum[31:24]),
    .Cout(),
    .P_block(P[3]),
    .G_block(G[3])
);

assign Cout = C[4];

endmodule
```

Listing 9: Carry Lookahead Adder

```

`timescale 1ns/1ps

module tb;

reg  [31:0] A, B;
reg         Cin;

wire [31:0] Sum;
wire        Cout;

cla32 DUT (
    .A(A),
    .B(B),
    .Cin(Cin),
    .Sum(Sum),
    .Cout(Cout)
);
```

```
19
20 initial begin
21     $dumpfile("dump.vcd");
22     $dumpvars(0,tb);
23
24     $display("
   ----------------------------------------------------------------
   ");
25     $display("Time\tA\t\tB\t\tCin\tSum\t\tCout");
26     $display("
   ----------------------------------------------------------------
   ");
27
28     A = 32'h00000000;
29     B = 32'h00000000;
30     Cin = 0;
31     #10;
32     $display("%0t\t%h\t%h\t%b\t%h\t%b",$time,A,B,Cin,Sum,Cout);
33
34     A = 32'h0000000F;
35     B = 32'h00000001;
36     Cin = 0;
37     #10;
38     $display("%0t\t%h\t%h\t%b\t%h\t%b",$time,A,B,Cin,Sum,Cout);
39
40     A = 32'h12345678;
41     B = 32'h87654321;
42     Cin = 0;
43     #10;
44     $display("%0t\t%h\t%h\t%b\t%h\t%b",$time,A,B,Cin,Sum,Cout);
45
46     A = 32'hFFFFFFFF;
47     B = 32'h00000001;
48     Cin = 0;
49     #10;
50     $display("%0t\t%h\t%h\t%b\t%h\t%b",$time,A,B,Cin,Sum,Cout);
51
52     A = 32'hAAAAAAAA;
53     B = 32'h55555555;
54     Cin = 1;
55     #10;
56     $display("%0t\t%h\t%h\t%b\t%h\t%b",$time,A,B,Cin,Sum,Cout);
57
58     #10;
59     $finish;
60 end
61
62 endmodule
```

Listing 10: Testbench

3.3 Simulation Result

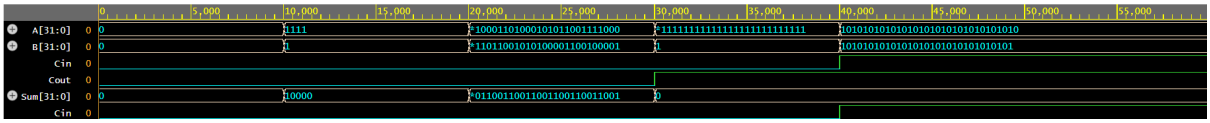

Figure 5: Simulation waveform for Carry Lookahead Adder

3.4 Performance Comparison

Ripple Carry Adders propagate carries through every stage.

Therefore,

$$Delay \propto n$$

where n is the number of bits.

Carry Lookahead Adders compute carries in parallel using generate and propagate signals.

Hence,

$$Delay \ll n$$

resulting in significantly faster operation.

3.5 Comparison Table

Feature	Ripple Carry Adder	Carry Lookahead Adder
Carry Generation	Sequential	Parallel
Speed	Lower	Higher
Propagation Delay	High	Low
Hardware Complexity	Simple	Higher
Area	Smaller	Larger
Maximum Clock Frequency	Lower	Higher

Table 1: Comparison of Ripple Carry and Carry Lookahead Adders

4 Discussion

The sequence detector successfully identifies the required pattern for both overlapping and non-overlapping cases.

The cascaded ripple carry adder is simple to design but has a relatively large propagation delay because each carry must propagate through every 8-bit block.

Introducing pipelining increases throughput by allowing different input pairs to be processed simultaneously.

The Carry Lookahead Adder further improves performance by computing carry signals in parallel, reducing the critical path delay and increasing the maximum operating frequency.

5 Conclusion

In this assignment, three digital designs were implemented using Verilog.

The sequence detector demonstrated finite state machine implementation for overlapping and non-overlapping sequence detection.

The cascaded 32-bit ripple carry adder illustrated hierarchical design, while pipelining improved throughput significantly.

Finally, the Carry Lookahead Adder achieved superior performance by reducing carry propagation delay, making it more suitable for high-speed arithmetic applications.